

CSCI 4160/6963, ECSE 4965/6965

Reinforcement Learning

Homework 2

Overview

This homework is a linear regression/classification assignment. You will train a linear classifier on the MNIST dataset, by fitting a line using least squares. Since regression works best when there are two classes only, you will apply regression to each pair of classes. MNIST has 10 classes (10 digits), so the total number of regressors you train will be $(10 \text{ choose } 2) = 45$. For each pair, fit the multidimensional regression as we discussed in class and then threshold the output so as to predict a class. For the threshold, you can either use 0.5 or you can experiment with other values and see how they affect the performance. **Graduate students are required to** implement the `get_optimal_thresh` function to search for the “optimal” threshold, e.g., by discretizing the space of all thresholds and finding the one that produces the best accuracy.

Logistics

All coding assignments in this course will be in Python. If you need help with Python, please talk to me ahead of time, so we can discuss the best way to get familiar with it.

For this assignment, you will be using your own computer, which should be sufficient. The code should not take more than 5 minutes to run on any standard laptop.

You are provided with skeleton starting code as well as with the MNIST dataset. To run the code, you will need a version of Python 3. Please use the provided `requirements.txt` file to ensure that everyone uses the same library versions. This will help avoid confusion due to your code potentially having different behavior when we run it on our machines. To install the libraries in the `requirements.txt` file, first set up a virtual environment (e.g., **conda**) and then run the following command:

```
pip install -r requirements.txt
```

Grading

For a full score, you will need to train all 45 pairs of regressions on the training set and achieve at least 90% accuracy on the test set for each one. You will lose 5 points for every percentage point below 90%. Your code must follow the provided skeleton code structure, in particular the `train` and `test` functions (as well as the `get_optimal_thresh` function, if you are a graduate student). You are not allowed to use a built-in pseudo-inverse function or a singular

value decomposition function; in other words, you have to implement the pseudo-inverse yourself (you can use matrix multiplication and inverse).

Make sure your code prints out all required information – we should not have to insert any print statements in order to find out whether your code works!

Finally, you must provide brief answers (2-3 sentences each) to the following questions:

- 1) What are two challenges with using K-Nearest-Neighbors on the MNIST dataset? If in doubt, you can implement it and see what happens.
- 2) Why does using a simple linear classifier work so well on MNIST?
- 3) How can you extend the linear classifier method to the multi-class case?

Hints and Tips

- The main challenge with this homework is stacking the data the correct way and inverting the correct matrix.
- Note that although the images are 28×28 , we are technically viewing them as vectors (in $28*28 = 784$ dimensions). Look at the provided `plot_image` function to see how to plot images and for an example manipulation.
- As we discussed in class, a matrix may be singular (i.e., not invertible) either because you stacked the data along the wrong dimension or because the data somehow resulted in a singular matrix. If the former happens, think about which dimension you should stack the data along so that you get a full-rank matrix. If the latter happens, think about what the data looks like and why vectors are not independent (then think about what would be a good way to make the matrix invertible without tampering with the data too much).
- If your code is running slowly, it could be due to the fact that numpy stored the matrices as integers. Converting them to floats should speed things up.

Submission

Please use LMS to submit a zip file containing 1) your **.py** code, along with instructions on how to run it, and 2) a **.pdf** file containing your answers to the above questions. The deadline is **11:59pm on Thursday, September 18.**